



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 4

по курсу «Разработка параллельных и распределенных программ»

Студент: Ивенкова О. В.

Группа: ИУ9-52Б

Преподаватель: Царёв А. С.

Москва, 2025

Содержание

1	Постановка задачи	3
2	Практическая реализация	4
3	Результат	11
4	Характеристики устройства	14

1 Постановка задачи

Лабораторная работа №4.

Задача о пяти обедающих философах.

Суть задачи следующая. Пять философов сидят за круглым столом. Они проводят жизнь, чередуя приёмы пищи и размышления. В центра стола находится большое блюдо спагетти. Чтобы съесть порцию, каждому философу нужно две вилки. Однако, вилок всего пять: между каждой парой рядом сидящих философов лежат по одной вилке, и каждый философ может пользоваться только теми вилками, которые лежат рядом с ним, слева и справа. Философ не может брать две вилки одновременно: сначала он тратит некоторое время на то, чтобы взять одну, затем вторую. Однако, он может одновременно положить их на место.

Задача заключается в том, чтобы написать программу, моделирующую поведение философов. Очевидно, что раз вилок всего пять, то одновременно есть могут не более двух философов, и два сидящих рядом философа не могут есть одновременно. Для имитации периодов раздумий и приёмов пищи можно использовать генератор случайных чисел, позволяющий задавать времена их действий в определённом интервале. Имитация поведения каждого философа, по сути, разбивается на то, что в любой момент времени философ находится в одном из пяти состояний: размышляет, берёт левую вилку, берёт правую вилку, ест, кладёт вилки на место. Таким образом, вилки являются разделяемым ресурсом.

На программу накладываются условия:

1. Каждый философ, по сути, является потоком, и модель поведения у каждого из них должна быть одинаковой, кроме того, какие вилки они могут брать.
2. Накладывание блокировки по сути является действием по взятию вилки, поэтому накладывать блокировку сразу на обе вилки нельзя; последовательность действий должна быть «наложить блокировку – взять вилку – наложить вторую блокировку – взять вторую вилку».
3. Программа должна избегать ситуации взаимоблокировки: ситуации, в которой все философы голодны, то есть ни один из них не может взять себе две вилки (например, когда каждый держит по одной и не хочет её отдавать).

Запрограммировать остановку алгоритма по достижении контрольного времени (например, атомарной операцией над булевым флагом). В отчёте построить некоторый результат работы алгоритма, которая может быть в виде графика, таблицы, лога или чего угодно ещё; главное условие состоит в том, чтобы по результатам можно было однозначно определить, чем в каждый момент времени был занят каждый философ (одно из пяти состояний).

Также рассмотреть вариант программы с увеличением количества философов до произвольного N .

2 Практическая реализация

Листинг 1: Код программы

```
1 #include <iostream>
2 #include <vector>
3 #include <thread>
4 #include <mutex>
5 #include <condition_variable>
6 #include <chrono>
7 #include <random>
8 #include <atomic>
9 #include <fstream>
10 #include <sstream>
11 #include <iomanip>
12
13 enum class State {
14     THINKING,
15     TAKING_LEFT,
16     TAKING_RIGHT,
17     EATING,
18     PUTTING_DOWN
19 };
20
21 static const char* stateToString(State s) {
22     switch (s) {
23         case State::THINKING: return "THINKING";
24         case State::TAKING_LEFT: return "TAKING_LEFT";
25         case State::TAKING_RIGHT: return "TAKING_RIGHT";
26         case State::EATING: return "EATING";
27         case State::PUTTING_DOWN: return "PUTTING_DOWN";
28         default: return "UNKNOWN";
29     }
```

```

30 }
31
32 class Waiter {
33 public:
34     explicit Waiter(size_t limit) : limit_(limit), counter_(0) {}
35
36     void acquire() {
37         std::unique_lock<std::mutex> lk(mtx_);
38         cv_.wait(lk, [this]() { return counter_ < limit_; });
39         ++counter_;
40     }
41
42     void release() {
43         {
44             std::lock_guard<std::mutex> lk(mtx_);
45             if (counter_ > 0) --counter_;
46         }
47         cv_.notify_one();
48     }
49
50 private:
51     size_t limit_;
52     size_t counter_;
53     std::mutex mtx_;
54     std::condition_variable cv_;
55 };
56
57 struct LogEntry {
58     long long time_ms;
59     size_t philosopher;
60     State state;
61 };
62
63 class Logger {
64 public:
65     void push(long long t, size_t id, State s) {
66         std::lock_guard<std::mutex> lk(mtx_);
67         log_.push_back({ t, id, s });
68     }
69
70     void toCSV(const std::string& filename) {
71         std::lock_guard<std::mutex> lk(mtx_);
72         std::ofstream ofs(filename);
73         ofs << "time_ms,philosopher,state\n";
74         for (auto& e : log_) {

```

```

75         ofs << e.time_ms << " " << e.philosopher << " " <<
stateToString(e.state) << "\n";
76     }
77 }
78
79     std::vector<size_t> countEats(size_t N) {
80         std::vector<size_t> cnt(N, 0);
81         std::lock_guard<std::mutex> lk(mtx_);
82         for (size_t i = 0; i + 1 < log_.size(); ++i) {
83             if (log_[i].state == State::EATING) ++cnt[log_[i].
philosopher];
84         }
85         return cnt;
86     }
87
88 private:
89     std::vector<LogEntry> log_;
90     std::mutex mtx_;
91 };
92
93 long long now_ms(const std::chrono::steady_clock::time_point& start) {
94     auto d = std::chrono::steady_clock::now() - start;
95     return std::chrono::duration_cast<std::chrono::milliseconds>(d).
count();
96 }
97
98 class PhilosophersMonitor {
99 private:
100     std::vector<State> states;
101     std::mutex mtx;
102     const std::chrono::steady_clock::time_point& start_time;
103     Logger& logger;
104
105     long long getCurrentTime() {
106         return now_ms(start_time);
107     }
108
109 public:
110     PhilosophersMonitor(size_t N, const std::chrono::steady_clock::
time_point& start, Logger& log_ref)
111         : states(N, State::THINKING), start_time(start), logger(log_ref)
{
112     }
113
114     void updateState(size_t id, State new_state) {
115         std::lock_guard<std::mutex> lock(mtx);

```

```

116
117     long long current_time = getCurrentTime();
118     logger.push(current_time, id, new_state);
119     states[id] = new_state;
120
121     printStates(current_time);
122 }
123
124 void printStates(long long current_time) {
125     std::cout << "\nTime: " << current_time << "ms\n";
126     std::cout << std::string(60, '-') << "\n";
127     std::cout << "| " << std::setw(10) << "Philosopher" << " | "
128         << std::setw(15) << "State" << " | "
129         << std::setw(25) << "Description" << " |\n";
130     std::cout << std::string(60, '-') << "\n";
131
132     for (size_t i = 0; i < states.size(); ++i) {
133         std::string description;
134         switch (states[i]) {
135             case State::THINKING:
136                 description = "Thinking deeply";
137                 break;
138             case State::TAKING_LEFT:
139                 description = "Taking left fork " + std::to_string(i);
140                 break;
141             case State::TAKING_RIGHT:
142                 description = "Taking right fork " + std::to_string((i +
143 1) % states.size());
144                 break;
145             case State::EATING:
146                 description = "Eating with forks " + std::to_string(i) +
147 " and " + std::to_string((i + 1) % states.size());
148                 break;
149             case State::PUTTING_DOWN:
150                 description = "Putting down both forks";
151                 break;
152             }
153
154         std::cout << "| " << std::setw(10) << i << " | "
155             << std::setw(15) << stateToString(states[i]) << " | "
156             << std::setw(25) << description << " |\n";
157     }
158
159     std::cout << std::string(60, '-') << std::endl;
160 }
161
162 std::vector<State> getCurrentStates() {

```

```

160         std::lock_guard<std::mutex> lock(mtx);
161         return states;
162     }
163 };
164
165 void philosopher_routine(size_t id,
166     size_t N,
167     std::vector<std::mutex>& forks,
168     Waiter& waiter,
169     PhilosophersMonitor& monitor,
170     std::atomic<bool>& stop_flag,
171     const std::chrono::steady_clock::time_point& start_time,
172     int think_min_ms, int think_max_ms,
173     int eat_min_ms, int eat_max_ms,
174     std::mt19937& rng_for_thread)
175 {
176     size_t left = id;
177     size_t right = (id + 1) % N;
178
179     std::uniform_int_distribution<int> think_dist(think_min_ms,
180     think_max_ms);
181     std::uniform_int_distribution<int> eat_dist(eat_min_ms, eat_max_ms);
182
183     while (!stop_flag.load(std::memory_order_relaxed)) {
184         monitor.updateState(id, State::THINKING);
185         std::this_thread::sleep_for(std::chrono::milliseconds(think_dist
186         (rng_for_thread)));
187         if (stop_flag.load(std::memory_order_relaxed)) break;
188
189         waiter.acquire();
190
191         monitor.updateState(id, State::TAKING_LEFT);
192         forks[left].lock();
193
194         monitor.updateState(id, State::TAKING_RIGHT);
195         forks[right].lock();
196
197         monitor.updateState(id, State::EATING);
198         std::this_thread::sleep_for(std::chrono::milliseconds(eat_dist(
199         rng_for_thread)));
200
201         monitor.updateState(id, State::PUTTING_DOWN);
202         forks[right].unlock();
203         forks[left].unlock();
204
205         waiter.release();

```



```

203     }
204 }
205
206 int main(int argc, char* argv[]) {
207     size_t N = 5;
208     int think_min_ms = 50;
209     int think_max_ms = 200;
210     int eat_min_ms = 50;
211     int eat_max_ms = 150;
212     int simulation_seconds = 10;
213     unsigned seed = (unsigned)std::chrono::steady_clock::now().
time_since_epoch().count();
214
215     std::cout << "Dining philosophers simulation\n";
216     std::cout << "N = " << N << ", simulation_seconds = " <<
simulation_seconds << ", seed = " << seed << "\n";
217     std::cout << "Initializing philosophers...\n\n";
218
219     std::vector<std::mutex> forks(N);
220     Waiter waiter((N > 1) ? (N / 2) : 1);
221     Logger logger;
222     std::atomic<bool> stop_flag(false);
223
224     auto start_time = std::chrono::steady_clock::now();
225     PhilosophersMonitor monitor(N, start_time, logger);
226
227     monitor.printStates(0);
228
229     std::random_device rd;
230     std::mt19937 seed_gen(seed);
231     std::vector<unsigned> thread_seeds(N);
232     for (size_t i = 0; i < N; ++i) thread_seeds[i] = seed_gen();
233
234     std::vector<std::thread> threads;
235     threads.reserve(N);
236     for (size_t i = 0; i < N; ++i) {
237         std::mt19937 rng(thread_seeds[i]);
238         threads.emplace_back(philosopher_routine,
239             i, N, std::ref(forks), std::ref(waiter), std::ref(monitor),
240             std::ref(stop_flag), std::cref(start_time),
241             think_min_ms, think_max_ms, eat_min_ms, eat_max_ms,
242             std::ref(rng));
243     }
244
245     std::this_thread::sleep_for(std::chrono::seconds(simulation_seconds)
);

```

```

246     stop_flag.store(true , std::memory_order_relaxed);
247
248     for (auto& t : threads) {
249         if (t.joinable()) t.join();
250     }
251
252     std::string csvname = "philosophers_log.csv";
253     logger.toCSV(csvname);
254
255     auto counts = logger.countEats(N);
256     std::cout << "\n\n=== SIMULATION COMPLETE ===\n";
257     std::cout << "Summary: number of 'EATING' events per philosopher:\n"
;
258     for (size_t i = 0; i < N; ++i) {
259         std::cout << "Philosopher " << std::setw(2) << i << ": " <<
counts[i] << " times\n";
260     }
261
262     std::cout << "Detailed log exported to " << csvname << "\n";
263     return 0;
264 }

```

3 Результат

Фрагмент результата, выведенного в консоль после запуска программы.

Time: 299ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>THINKING</i>	<i>Thinking deeply</i>
1	<i>EATING</i>	<i>Eating with forks 1 and 2</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>THINKING</i>	<i>Thinking deeply</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 308ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>TAKING_LEFT</i>	<i>Taking left fork 0</i>
1	<i>EATING</i>	<i>Eating with forks 1 and 2</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>THINKING</i>	<i>Thinking deeply</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 316ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>TAKING_RIGHT</i>	<i>Taking right fork 1</i>
1	<i>EATING</i>	<i>Eating with forks 1 and 2</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>THINKING</i>	<i>Thinking deeply</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 451ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>TAKING_RIGHT</i>	<i>Taking right fork 1</i>
1	<i>PUTTING_DOWN</i>	<i>Putting down both forks</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>THINKING</i>	<i>Thinking deeply</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 473ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>TAKING_RIGHT</i>	<i>Taking right fork 1</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>THINKING</i>	<i>Thinking deeply</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 490ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>EATING</i>	<i>Eating with forks 0 and 1</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>THINKING</i>	<i>Thinking deeply</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 499ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>EATING</i>	<i>Eating with forks 0 and 1</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>TAKING_LEFT</i>	<i>Taking left fork 3</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 507ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>EATING</i>	<i>Eating with forks 0 and 1</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>TAKING_RIGHT</i>	<i>Taking right fork 4</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 514ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>EATING</i>	<i>Eating with forks 0 and 1</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>EATING</i>	<i>Eating with forks 3 and 4</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 626ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>PUTTING_DOWN</i>	<i>Putting down both forks</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>EATING</i>	<i>Eating with forks 3 and 4</i>
4	<i>THINKING</i>	<i>Thinking deeply</i>

Time: 635ms

<i>Philosopher</i>	<i>State</i>	<i>Description</i>
0	<i>PUTTING_DOWN</i>	<i>Putting down both forks</i>
1	<i>THINKING</i>	<i>Thinking deeply</i>
2	<i>THINKING</i>	<i>Thinking deeply</i>
3	<i>EATING</i>	<i>Eating with forks 3 and 4</i>
4	<i>TAKING_LEFT</i>	<i>Taking left fork 4</i>

4 Характеристики устройства

Имя устройства Acer Extensa 215-54

Оперативная память 16,0 ГБ

Процессор устройства Intel Core i5-1135G7 (2.40 ГГц, 4 ядра, 15 Вт)